PROJECT MILESTONE 1

Kalman Filter for 3D Motion Tracking

Advanced State Estimation Techniques

Team Name: WARJINGO

Github Repository:

https://github.com/Arham8bit/CAAL_S26_Project_KalmanFilter_Warjingo/tree/main

Team Members

Member Name	ERP
Muhammad Ismail	31689
Mehdi Ali	30507
Muhammad Ismail	30917
Arham Awan	30934

TABLE OF CONTENTS

Contents

1 State Definition

1.1 Purpose of the State

The objective of this project is to estimate the motion of an object moving in three-dimensional space. Since the system cannot directly measure all motion quantities and sensor measurements are affected by noise, a complete system state is defined to represent the object's true motion at any time.

1.2 State Vector

The system state includes motion along the x, y, and z axes, with four motion variables per axis:

$$x = \begin{bmatrix} p_x & v_x & a_x & j_x & p_y & v_y & a_y & j_y & p_z & v_z & a_z & j_z \end{bmatrix}^T \tag{1}$$

This results in a **12-dimensional state vector**, where:

- p represents **position**
- v represents **velocity**
- a represents **acceleration**
- j represents **jerk**

1.3 Physical Meaning of State Variables

For each spatial axis, the state variables have the following meanings:

Position $p(t)$:
Represents the spatial location of the object.

Velocity $v(t)$:
Describes how fast the position is changing with time.

Acceleration $a(t)$:
Describes how the velocity is changing, capturing speeding up or slowing down.

Jerk $j(t)$:
Describes how acceleration changes over time and controls the smoothness of motion.

1.4 Mathematical Relationships

The motion variables follow standard kinematic relationships:

$$v(t) = \frac{dp(t)}{dt} \tag{2}$$

$$a(t) = \frac{dv(t)}{dt} = \frac{d^2p(t)}{dt^2} \tag{3}$$

$$j(t) = \frac{da(t)}{dt} = \frac{d^3p(t)}{dt^3} \tag{4}$$

These equations show the hierarchical relationship between jerk, acceleration, velocity, and position.

1.5 Justification for Including Jerk

Including jerk in the state model improves estimation accuracy and realism. Models that exclude jerk often produce sudden changes in acceleration, which do not represent real-world motion accurately. By assuming constant jerk over small time intervals, the model is able to capture smooth yet rapidly changing motion. This provides better predictions while keeping the model mathematically simple and suitable for Kalman filtering.

2 Continuous-Time Motion Model

2.1 Modeling Assumption

The motion of the object is modeled under the assumption of constant jerk over small time intervals. This means that while acceleration is allowed to change, the rate at which acceleration changes is assumed to remain constant. This assumption helps in modeling realistic motion without making the system too complex.

2.2 Explanation of the Motion Model

The first equation states that velocity is the rate of change of position. The second equation shows that acceleration is the rate of change of velocity. The third equation indicates that jerk controls how acceleration changes. The final equation enforces the assumption that jerk remains constant.

Together, these equations form a chain where jerk affects acceleration, acceleration affects velocity, and velocity affects position. This results in smooth and continuous motion.

2.3 Why Constant Jerk is a Reasonable Assumption

In many real-world systems, motion does not change suddenly. Physical limitations prevent instant changes in acceleration. Examples include vehicle motion, drone movement, and camera tracking systems. In such cases, acceleration changes gradually, making the constant jerk assumption reasonable over short time intervals.

This assumption also keeps the system linear, which is important for applying Kalman filtering efficiently.

2.4 Model Limitations

The constant jerk model may not be suitable for situations involving sudden impacts or abrupt control actions. However, when small time steps are used, it provides a good approximation of smooth motion and works well for estimation purposes.

3 Discrete-Time State-Space Model

3.1 Why we need a discrete-time model

In real systems:

- Measurements arrive at fixed time intervals
- The Kalman Filter works in discrete time
- So we must convert continuous equations into time-step equations

Let:

- Current time $= t$
- Next time $= t + \Delta t$

3.2 State variables (one axis)

For simplicity, consider one axis only (x-axis). The same logic applies to y and z.

- $p(t) \rightarrow$ position
- $v(t) \rightarrow$ velocity
- $a(t) \rightarrow$ acceleration
- $j(t) \rightarrow$ jerk

Assumption: Jerk is constant over the interval Δt

3.3 Tool used: Taylor Series Expansion

General Taylor expansion of a function:

$$f(t + \Delta t) = f(t) + f'(t)\Delta t + \frac{f''(t)}{2!}(\Delta t)^2 + \frac{f'''(t)}{3!}(\Delta t)^3 + \cdots \tag{5}$$

We will apply this one state at a time.

3.3.1 1. Position equation

Start with position $p(t)$.

Taylor expansion of position:

$$p(t + \Delta t) = p(t) + p'(t)\Delta t + \frac{p''(t)}{2}(\Delta t)^2 + \frac{p'''(t)}{6}(\Delta t)^3 \tag{6}$$

Now substitute derivatives:

- $p'(t) = v(t)$
- $p''(t) = a(t)$
- $p'''(t) = j(t)$

Final discrete position equation:

$$p(t + \Delta t) = p(t) + v(t)\Delta t + \frac{a(t)}{2}(\Delta t)^2 + \frac{j(t)}{6}(\Delta t)^3 \tag{7}$$

3.3.2 2. Velocity equation

Taylor expansion of velocity $v(t)$:

$$v(t+\Delta t) = v(t) + v'(t)\Delta t + \frac{v''(t)}{2}(\Delta t)^2 \tag{8}$$

Substitute derivatives:

- $v'(t) = a(t)$
- $v''(t) = j(t)$

Final discrete velocity equation:

$$v(t+\Delta t) = v(t) + a(t)\Delta t + \frac{j(t)}{2}(\Delta t)^2 \tag{9}$$

3.3.3 3. Acceleration equation

Taylor expansion of acceleration $a(t)$:

$$a(t+\Delta t) = a(t) + a'(t)\Delta t \tag{10}$$

Substitute derivative:

- $a'(t) = j(t)$

Final discrete acceleration equation:

$$a(t+\Delta t) = a(t) + j(t)\Delta t \tag{11}$$

3.3.4 4. Jerk equation

Since jerk is assumed constant:

$$j(t+\Delta t) = j(t) \tag{12}$$

So:

$$j(t+\Delta t) = j(t) \tag{13}$$

3.4 Derivation of the State Transition Matrix F

3.4.1 What F represents (simple words)

The state transition matrix F tells us:
How the state at time k produces the state at time k+1
Mathematically:

$$x_{k+1} = Fx_k \tag{14}$$

So each row of F comes directly from the discrete equations we already derived.

3.4.2 Step 1: Define the state vector

The full system tracks motion in three independent axes (x, y, z).

The state vector is:

$$x = \begin{bmatrix} p_x \\ v_x \\ a_x \\ j_x \\ p_y \\ v_y \\ a_y \\ j_y \\ p_z \\ v_z \\ a_z \\ j_z \end{bmatrix} \tag{15}$$

This is a 12×1 vector, so F must be 12×12.

3.4.3 Step 2: Start with ONE axis

Because the axes are independent, first derive F for one axis.

Let the one-axis state be:

$$x_{axis} = \begin{bmatrix} p \\ v \\ a \\ j \end{bmatrix} \tag{16}$$

3.4.4 Step 3: Write the discrete equations again

From the previous derivation:

$$p(t + \Delta t) = p(t) + v(t)\Delta t + \frac{a(t)}{2}(\Delta t)^2 + \frac{j(t)}{6}(\Delta t)^3 \tag{17}$$

$$v(t + \Delta t) = v(t) + a(t)\Delta t + \frac{j(t)}{2}(\Delta t)^2 \tag{18}$$

$$a(t + \Delta t) = a(t) + j(t)\Delta t \tag{19}$$

$$j(t + \Delta t) = j(t) \tag{20}$$

3.4.5 Step 4: Convert equations into matrix form

We now express each equation as a row of a matrix.

Row 1: Position equation

$$p(t + \Delta t) = 1 \cdot p(t) + \Delta t \cdot v(t) + \frac{(\Delta t)^2}{2} \cdot a(t) + \frac{(\Delta t)^3}{6} \cdot j(t) \tag{21}$$

So the first row is:

$$\begin{bmatrix} 1 & \Delta t & \frac{(\Delta t)^2}{2} & \frac{(\Delta t)^3}{6} \end{bmatrix} \tag{22}$$

Row 2: Velocity equation

$$v(t+\Delta t) = 0 \cdot p(t) + 1 \cdot v(t) + \Delta t \cdot a(t) + \frac{(\Delta t)^2}{2} \cdot j(t) \tag{23}$$

Row:

$$\begin{bmatrix} 0 & 1 & \Delta t & \frac{(\Delta t)^2}{2} \end{bmatrix} \tag{24}$$

Row 3: Acceleration equation

$$a(t+\Delta t) = 0 \cdot p(t) + 0 \cdot v(t) + 1 \cdot a(t) + \Delta t \cdot j(t) \tag{25}$$

Row:

$$\begin{bmatrix} 0 & 0 & 1 & \Delta t \end{bmatrix} \tag{26}$$

Row 4: Jerk equation

$$j(t+\Delta t) = 0 \cdot p(t) + 0 \cdot v(t) + 0 \cdot a(t) + 1 \cdot j(t) \tag{27}$$

Row:

$$\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} \tag{28}$$

3.4.6 Step 5: One-axis state transition matrix

Putting the rows together:

$$F_{axis} = \begin{bmatrix} 1 & \Delta t & \frac{(\Delta t)^2}{2} & \frac{(\Delta t)^3}{6} \\ 0 & 1 & \Delta t & \frac{(\Delta t)^2}{2} \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{29}$$

This matrix describes how one axis evolves over time.

3.4.7 Step 6: Extend to 3D (12 × 12 matrix)

Since:

- x, y, z axes do not affect each other
- motion in each axis follows the same model

The full matrix is block-diagonal:

$$F = \begin{bmatrix} F_{axis} & 0 & 0 \\ 0 & F_{axis} & 0 \\ 0 & 0 & F_{axis} \end{bmatrix} \tag{30}$$

Each block is 4 × 4, so total size is 12 × 12.

3.4.8 Step 7: full 12 × 12 matrix explicitly

graphicx

$$F = \begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 & \frac{1}{6}\Delta t^3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & \Delta t & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 & \frac{1}{6}\Delta t^3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & \Delta t & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 & \frac{1}{6}\Delta t^3 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

(31)

4 Derivation of Noise Input Matrix G and Process Noise Covariance Matrix Q

4.1 Why noise is included in the model

In the constant jerk motion model, jerk is assumed to be constant over a short time interval.

However, in real physical systems, motion is never perfectly predictable. Small unmodeled forces, disturbances, and modeling errors introduce randomness into the jerk.

To capture this uncertainty, jerk is modeled as:

deterministic jerk + random jerk noise

This random component is treated as zero-mean white noise.

4.2 Definition of jerk noise

Let:

- w_k be the discrete-time jerk noise
- Mean: $E[w_k] = 0$
- Variance: $E[w_k^2] = \sigma_j^2$

This noise affects the system only through the jerk state.

4.3 Step 1: Discrete-time equations including noise (one axis)

For one axis (x-axis), the state vector is:

$$x_k = \begin{bmatrix} p_k \\ v_k \\ a_k \\ j_k \end{bmatrix} \tag{32}$$

The discrete-time equations (derived using Taylor series) are:

$$p_{k+1} = p_k + v_k \Delta t + \frac{1}{2} a_k \Delta t^2 + \frac{1}{6} j_k \Delta t^3 \tag{33}$$

$$v_{k+1} = v_k + a_k \Delta t + \frac{1}{2} j_k \Delta t^2 \tag{34}$$

$$a_{k+1} = a_k + j_k \Delta t \tag{35}$$

$$j_{k+1} = j_k \tag{36}$$

To include noise, jerk is replaced by:

$$j_k \to j_k + w_k \tag{37}$$

4.4 Step 2: Substitute noise into each equation

4.4.1 Position

$$p_{k+1} = p_k + v_k \Delta t + \frac{1}{2} a_k \Delta t^2 + \frac{1}{6} (j_k + w_k) \Delta t^3 \tag{38}$$

Noise contribution:

$$\frac{1}{6} \Delta t^3 w_k \tag{39}$$

4.4.2 Velocity

$$v_{k+1} = v_k + a_k \Delta t + \frac{1}{2} (j_k + w_k) \Delta t^2 \tag{40}$$

Noise contribution:

$$\frac{1}{2} \Delta t^2 w_k \tag{41}$$

4.4.3 Acceleration

$$a_{k+1} = a_k + (j_k + w_k) \Delta t \tag{42}$$

Noise contribution:

$$\Delta t \, w_k \tag{43}$$

4.4.4 Jerk

$$j_{k+1} = j_k + w_k \tag{44}$$

Noise contribution:

$$1 \cdot w_k \tag{45}$$

4.5 Step 3: Write noise effect in matrix form

Collecting all noise terms:

$$\begin{bmatrix} p_{k+1} \\ v_{k+1} \\ a_{k+1} \\ j_{k+1} \end{bmatrix} = F \begin{bmatrix} p_k \\ v_k \\ a_k \\ j_k \end{bmatrix} + \begin{bmatrix} \frac{1}{6}\Delta t^3 \\ \frac{1}{2}\Delta t^2 \\ \Delta t \\ 1 \end{bmatrix} w_k \tag{46}$$

4.6 Step 4: Definition of G (one axis)

By comparison with the standard state equation:

$$x_{k+1} = Fx_k + Gw_k \tag{47}$$

The noise input matrix G is:

$$G_{axis} = \begin{bmatrix} \frac{1}{6}\Delta t^3 \\ \frac{1}{2}\Delta t^2 \\ \Delta t \\ 1 \end{bmatrix} \tag{48}$$

This matrix shows how jerk noise propagates into each state.

4.7 Step 5: Extension to 3D Motion

Each axis has independent jerk noise:

$$w_k = \begin{bmatrix} w_{x,k} \\ w_{y,k} \\ w_{z,k} \end{bmatrix} \tag{49}$$

where:

- $w_{x,k}$ = jerk noise in x-direction at time k
- $w_{y,k}$ = jerk noise in y-direction at time k
- $w_{z,k}$ = jerk noise in z-direction at time k

The full G matrix becomes block-diagonal:

$$G = \begin{bmatrix} G_{axis} & 0 & 0 \\ 0 & G_{axis} & 0 \\ 0 & 0 & G_{axis} \end{bmatrix} \tag{50}$$

This results in a 12×3 matrix.
Why 12×3?

- 12 rows: One for each state $(p_x, v_x, a_x, j_x, p_y, v_y, a_y, j_y, p_z, v_z, a_z, j_z)$
- 3 columns: One for each noise source (w_x, w_y, w_z)

4.8 Step 6: Definition of Process Noise Covariance Matrix Q

4.8.1 Mathematical Derivation of $Q = E[Gw_k w_k^T G^T]$

1. State equation with noise:

$$x_{k+1} = Fx_k + Gw_k \tag{51}$$

The term Gw_k represents how noise affects the state.

2. Process noise covariance definition:

The covariance matrix Q describes the uncertainty added to the state by the noise:

$$Q = \text{Cov}(Gw_k) = E[(Gw_k)(Gw_k)^T] \tag{52}$$

3. Expand the expression:

$$(Gw_k)(Gw_k)^T = (Gw_k)(w_k^T G^T) = Gw_k w_k^T G^T \tag{53}$$

(Matrix multiplication is associative)

4. Take expected value:

$$Q = E[Gw_k w_k^T G^T] \tag{54}$$

5. Key insight: G is a constant matrix (no randomness), so we can move it outside the expectation:

$$Q = GE[w_k w_k^T]G^T \tag{55}$$

This is valid because for constant matrices A and random vector x:

$$E[Ax] = AE[x] \quad \text{and} \quad E[xA^T] = E[x]A^T \tag{56}$$

4.9 Derivation of $E[w_k w_k^T] = \sigma_j^2 I$

Let's compute this step by step:

1. Write $w_k w_k^T$ explicitly:

$$w_k w_k^T = \begin{bmatrix} w_{x,k} \\ w_{y,k} \\ w_{z,k} \end{bmatrix} \begin{bmatrix} w_{x,k} & w_{y,k} & w_{z,k} \end{bmatrix} = \begin{bmatrix} w_{x,k}^2 & w_{x,k}w_{y,k} & w_{x,k}w_{z,k} \\ w_{y,k}w_{x,k} & w_{y,k}^2 & w_{y,k}w_{z,k} \\ w_{z,k}w_{x,k} & w_{z,k}w_{y,k} & w_{z,k}^2 \end{bmatrix} \tag{57}$$

2. Take expected value of each element:

Diagonal elements (variances):

$$E[w_{x,k}^2] = \sigma_{j_x}^2 \quad \text{(variance of x-jerk noise)} \tag{58}$$
$$E[w_{y,k}^2] = \sigma_{j_y}^2 \quad \text{(variance of y-jerk noise)} \tag{59}$$
$$E[w_{z,k}^2] = \sigma_{j_z}^2 \quad \text{(variance of z-jerk noise)} \tag{60}$$

Off-diagonal elements (covariances):

Assuming independent noise across axes (reasonable for 3D motion):

$$E[w_{x,k}w_{y,k}] = E[w_{x,k}]E[w_{y,k}] = 0 \times 0 = 0 \tag{61}$$
$$E[w_{x,k}w_{z,k}] = 0 \tag{62}$$
$$E[w_{y,k}w_{z,k}] = 0 \tag{63}$$

(We assume zero-mean noise: $E[w_{x,k}] = E[w_{y,k}] = E[w_{z,k}] = 0$)

3. Assumption: Equal jerk noise variance in all axes:

$$\sigma_{j_x}^2 = \sigma_{j_y}^2 = \sigma_{j_z}^2 = \sigma_j^2 \tag{64}$$

4. Final result:

$$E[w_k w_k^T] = \begin{bmatrix} \sigma_j^2 & 0 & 0 \\ 0 & \sigma_j^2 & 0 \\ 0 & 0 & \sigma_j^2 \end{bmatrix} = \sigma_j^2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \sigma_j^2 I_{3\times 3} \tag{65}$$

5. Substitute into Q expression:

$$Q = G(\sigma_j^2 I)G^T = \sigma_j^2 GG^T \tag{66}$$

Since $G(\sigma_j^2 I) = \sigma_j^2 G$ (scalar multiplication distributes)

4.10 Physical Interpretation

- $w_k w_k^T$: Represents all possible products of noise components
- $E[w_k w_k^T]$: Average behavior of noise (variances on diagonal, covariances off-diagonal)
- $\sigma_j^2 I$: Diagonal matrix means independent noise in each axis with equal strength

4.11 Step 7: Q Matrix for One Axis (4 × 4)

4.11.1 Derivation of $Q_{axis} = \sigma_j^2 G_{axis} G_{axis}^T$

1. For one axis only, the noise is scalar w_j (not vector):

2. Apply the general formula $Q = GE[ww^T]G^T$:

For scalar noise w_j:

- $G = G_{axis}$ (4×1 matrix)

$$G_{axis} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix} = \begin{bmatrix} \frac{\Delta t^3}{6} \\ \frac{\Delta t^2}{2} \\ \Delta t \\ 1 \end{bmatrix} \tag{67}$$

$$E[w_j w_j^T] = E[w_j^2] = \sigma_j^2 \quad \text{(scalar variance)} \tag{68}$$

So:

$$Q_{axis} = G_{axis}\sigma_j^2 G_{axis}^T = \sigma_j^2 G_{axis} G_{axis}^T \tag{69}$$

We multiply a 4×1 vector by a 1×4 vector, producing a 4×4 matrix.

$$GG^T = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix} \begin{bmatrix} g_1 & g_2 & g_3 & g_4 \end{bmatrix} \tag{70}$$

Where:

- $g_1 = \frac{\Delta t^3}{6}$
- $g_2 = \frac{\Delta t^2}{2}$
- $g_3 = \Delta t$
- $g_4 = 1$

Each matrix element is computed as:

$$Q_{ij} = g_i \times g_j \tag{71}$$

Row 1

$$Q_{11} = g_1 g_1 = \frac{1}{6}\Delta t^3 \cdot \frac{1}{6}\Delta t^3 = \frac{\Delta t^6}{36} \tag{72}$$

$$Q_{12} = g_1 g_2 = \frac{1}{6}\Delta t^3 \cdot \frac{1}{2}\Delta t^2 = \frac{\Delta t^5}{12} \tag{73}$$

$$Q_{13} = g_1 g_3 = \frac{1}{6}\Delta t^3 \cdot \Delta t = \frac{\Delta t^4}{6} \tag{74}$$

$$Q_{14} = g_1 g_4 = \frac{1}{6}\Delta t^3 \tag{75}$$

Row 2

$$Q_{21} = g_2 g_1 = \frac{\Delta t^5}{12} \tag{76}$$

$$Q_{22} = g_2 g_2 = \frac{1}{2}\Delta t^2 \cdot \frac{1}{2}\Delta t^2 = \frac{\Delta t^4}{4} \tag{77}$$

$$Q_{23} = g_2 g_3 = \frac{1}{2}\Delta t^2 \cdot \Delta t = \frac{\Delta t^3}{2} \tag{78}$$

$$Q_{24} = g_2 g_4 = \frac{1}{2}\Delta t^2 \tag{79}$$

Row 3

$$Q_{31} = g_3 g_1 = \frac{\Delta t^4}{6} \tag{80}$$

$$Q_{32} = g_3 g_2 = \frac{\Delta t^3}{2} \tag{81}$$

$$Q_{33} = g_3 g_3 = (\Delta t)^2 = \Delta t^2 \tag{82}$$

$$Q_{34} = g_3 g_4 = \Delta t \tag{83}$$

Row 4

$$Q_{41} = g_4 g_1 = \frac{\Delta t^3}{6} \tag{84}$$

$$Q_{42} = g_4 g_2 = \frac{1}{2}\Delta t^2 \tag{85}$$

$$Q_{43} = g_4 g_3 = \Delta t \tag{86}$$

$$Q_{44} = g_4 g_4 = 1 \tag{87}$$

4.12 Step 8: Final one-axis covariance matrix

Now multiply the full matrix by σ_j^2:

$$Q_{axis} = \sigma_j^2 \begin{bmatrix} \frac{\Delta t^6}{36} & \frac{\Delta t^5}{12} & \frac{\Delta t^4}{6} & \frac{\Delta t^3}{6} \\ \frac{\Delta t^5}{12} & \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^4}{6} & \frac{\Delta t^3}{2} & \Delta t^2 & \Delta t \\ \frac{\Delta t^3}{6} & \frac{\Delta t^2}{2} & \Delta t & 1 \end{bmatrix} \tag{88}$$

This matrix is symmetric, as required for any covariance matrix.

4.13 Step 9: Full 12 × 12 Q matrix (3D motion)

Because:

- x, y, z axes are independent
- each axis has identical noise behavior

The full process noise covariance matrix is block diagonal:

$$Q = \begin{bmatrix} Q_{axis} & 0 & 0 \\ 0 & Q_{axis} & 0 \\ 0 & 0 & Q_{axis} \end{bmatrix} \tag{89}$$

Let's assume:

- $\sigma_j^2 = \sigma^2$ (jerk noise variance, same for all axes)
- $\Delta t = T$

$$\mathrm{Q}_{12\times 12} = \sigma^2 \begin{bmatrix} \frac{T^6}{36} & \frac{T^5}{12} & \frac{T^4}{6} & \frac{T^3}{6} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{T^5}{12} & \frac{T^4}{4} & \frac{T^3}{2} & \frac{T^2}{2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{T^4}{6} & \frac{T^3}{2} & T^2 & T & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{T^3}{6} & \frac{T^2}{2} & T & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{T^6}{36} & \frac{T^5}{12} & \frac{T^4}{6} & \frac{T^3}{6} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{T^5}{12} & \frac{T^4}{4} & \frac{T^3}{2} & \frac{T^2}{2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{T^4}{6} & \frac{T^3}{2} & T^2 & T & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{T^3}{6} & \frac{T^2}{2} & T & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{T^6}{36} & \frac{T^5}{12} & \frac{T^4}{6} & \frac{T^3}{6} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{T^5}{12} & \frac{T^4}{4} & \frac{T^3}{2} & \frac{T^2}{2} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{T^4}{6} & \frac{T^3}{2} & T^2 & T \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{T^3}{6} & \frac{T^2}{2} & T & 1 \end{bmatrix}$$

5 Derivation of Matrices

State Transition Matrix F

We derived F by converting the continuous-time motion equations (position, velocity, acceleration, jerk) into discrete-time form using a fixed sampling interval Δt.

Each state at the next step $(t + \Delta t)$ is written in terms of the current states. This gives the full 12×12 F matrix showing how position, velocity, acceleration, and jerk evolve over time.

Noise Input Matrix G

We derived G by looking at how jerk noise affects all other states. Noise enters at the jerk level and integrates into acceleration, velocity, and position. That's why each row has powers of Δt (Δt, Δt^2, Δt^3).

Process Noise Covariance Q

The Q matrix is computed as:

$$Q = G\,\Sigma\,G^T \tag{90}$$

where Σ is the variance of jerk noise (σ_j^2). This shows how randomness in jerk spreads into all states.

Structure and Sparsity

- **Block-diagonal:** x, y, and z axes are independent, so each axis has its own block.
- **Sparse:** Many entries are zero because each state is affected only by related states.
- **Pattern:** Higher powers of Δt appear for position, lower powers for acceleration and jerk — reflecting how noise accumulates over time.

Physical Meaning

- F **matrix** $\rightarrow$ Shows deterministic motion: how the system moves if there is no noise.
- G **matrix** $\rightarrow$ Shows where the noise enters: only at the jerk level.
- Q **matrix** $\rightarrow$ Shows how uncertainty grows: position is most affected over time, jerk has the smallest uncertainty.
- **Intuition:** Bigger Δt or bigger σ_j^2 $\rightarrow$ bigger uncertainty. Makes sense physically: errors grow with time.

6 Measurement Model Derivation for Linear Kalman Filter

6.1 Measurement Definition

In the constant jerk motion model, it is assumed that the sensor system provides direct measurements of the object's position in three-dimensional space. This assumption is realistic for common tracking systems such as GPS, radar, and optical sensors, which primarily measure spatial coordinates rather than velocity or acceleration.

The measurement vector at discrete time step n is defined as:

$$\mathbf{z}_n = \begin{bmatrix} p_x(n) \\ p_y(n) \\ p_z(n) \end{bmatrix} \in \mathbb{R}^{3\times 1}$$

where:

- $p_x(n)$, $p_y(n)$, and $p_z(n)$ represent the measured Cartesian position coordinates along the x, y, and z axes at time step n.

6.2 State-to-Measurement Relationship

The complete system state vector consists of 12 variables describing motion along three axes:

$$\mathbf{x} = \begin{bmatrix} p_x \\ v_x \\ a_x \\ j_x \\ p_y \\ v_y \\ a_y \\ j_y \\ p_z \\ v_z \\ a_z \\ j_z \end{bmatrix} \in \mathbb{R}^{12\times 1}$$

For a linear system, the relationship between the state and the measurement is given by the standard Linear Kalman Filter measurement equation:

$$\mathbf{z}_n = \mathbf{H}\mathbf{x}_n + \mathbf{v}_n$$

where:

- $\mathbf{H} \in \mathbb{R}^{3\times 12}$ is the measurement matrix
- $\mathbf{v}_n \in \mathbb{R}^{3\times 1}$ represents measurement noise

In the ideal noise-free case, this equation reduces to:

$$\mathbf{z}_n = \mathbf{H}\mathbf{x}_n$$

6.3 Derivation of the Measurement Matrix H

Since the sensor measures only position, the measurement matrix must extract:

- p_x from the x-axis states
- p_y from the y-axis states
- p_z from the z-axis states

This requirement can be written as:

$$\begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} = \mathbf{H} \begin{bmatrix} p_x \\ v_x \\ a_x \\ j_x \\ p_y \\ v_y \\ a_y \\ j_y \\ p_z \\ v_z \\ a_z \\ j_z \end{bmatrix}$$

From the structure of the state vector:

- p_x is the 1st element
- p_y is the 5th element
- p_z is the 9th element

Therefore:

- A value of 1 is placed at these positions
- All other entries are set to 0

6.4 Measurement Matrix H

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

6.5 Verification of the Measurement Model

To verify the correctness of the measurement matrix, we multiply **H** with the state vector:

$$\mathbf{z}_n = \mathbf{H}\mathbf{x}_n$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} p_x \\ v_x \\ a_x \\ j_x \\ p_y \\ v_y \\ a_y \\ j_y \\ p_z \\ v_z \\ a_z \\ j_z \end{bmatrix} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

The result matches the measurement vector definition exactly, confirming the correctness of **H**.

7 Linear Kalman Filter Equations

7.1 Introduction to the Algorithm

The Linear Kalman Filter (LKF) is a recursive state estimation algorithm that operates in two main stages: prediction and update. In the prediction stage, the filter uses a mathematical model of the system to estimate the future state. In the update stage, this prediction is corrected using newly available measurements. By repeatedly alternating between these two steps, the Kalman Filter produces an accurate estimate of the system state even in the presence of noise and uncertainty.

7.2 Prediction Step

Equation (9): State Prediction

$$\hat{\mathbf{x}}_{n|n-1} = \mathbf{F}\hat{\mathbf{x}}_{n-1|n-1}$$

Purpose:

This equation predicts the state of the object at the next time step before any new measurement is used.

Explanation:

- $\hat{\mathbf{x}}_{n-1|n-1}$ represents the best estimate of the system state at time $n-1$.
- $\mathbf{F}$ is the state transition matrix, which models how the system evolves over one time step.
- Multiplying the previous state estimate by $\mathbf{F}$ propagates the state forward by Δt seconds according to the assumed motion model.
- $\hat{\mathbf{x}}_{n|n-1}$ is the predicted state estimate at time n, based solely on the model.

Example:

If at time $n-1$ an object is estimated to be at position $p = 100\,\text{m}$ with velocity $v = 20\,\text{m/s}$, and $\Delta t = 1\,\text{s}$, then the predicted position at time n becomes:

$$p = 100 + 20 = 120 \text{ m}$$

Equation (10): Uncertainty Prediction

$$\mathbf{P}_{n|n-1} = \mathbf{F}\mathbf{P}_{n-1|n-1}\mathbf{F}^T + \mathbf{Q}$$

Purpose:

This equation predicts the uncertainty associated with the predicted state.

Explanation:

- $\mathbf{P}_{n-1|n-1}$ is the uncertainty (covariance) of the previous state estimate.
- The term $\mathbf{F}\mathbf{P}_{n-1|n-1}\mathbf{F}^T$ propagates this uncertainty forward through the system dynamics.
- $\mathbf{Q}$ is the process noise covariance matrix, which accounts for uncertainties due to unmodeled effects such as disturbances or modeling errors.
- $\mathbf{P}_{n|n-1}$ is the predicted uncertainty before incorporating the new measurement.

Why both terms matter:

- The first term reflects the natural growth of uncertainty over time.
- The second term captures additional uncertainty introduced by random or unknown influences.

7.3 Update Step

Equation (11): Kalman Gain

$$\mathbf{K}_n = \mathbf{P}_{n|n-1}\mathbf{H}^T \left(\mathbf{H}\mathbf{P}_{n|n-1}\mathbf{H}^T + \mathbf{R}\right)^{-1}$$

Purpose:

The Kalman Gain determines how much weight should be given to the new measurement compared to the predicted state.

Explanation:

- $\mathbf{H}$ is the measurement matrix, which maps the state variables into measurement space.
- $\mathbf{R}$ is the measurement noise covariance matrix, representing sensor accuracy.
- $\mathbf{H}\mathbf{P}_{n|n-1}\mathbf{H}^T$ gives the predicted uncertainty of the measurement.
- The denominator $\left(\mathbf{H}\mathbf{P}_{n|n-1}\mathbf{H}^T + \mathbf{R}\right)$ represents the total expected measurement uncertainty.
- $\mathbf{K}_n$ is the optimal gain that minimizes the estimation error.

Interpretation:

- If the measurement noise $\mathbf{R}$ is small, the Kalman Gain increases, meaning the filter relies more on measurements.
- If the predicted uncertainty $\mathbf{P}_{n|n-1}$ is small, the Kalman Gain decreases, meaning the filter relies more on the model.

Equation (12): State Update

$$\hat{\mathbf{x}}_{n|n} = \hat{\mathbf{x}}_{n|n-1} + \mathbf{K}_n \left(\mathbf{z}_n - \mathbf{H}\hat{\mathbf{x}}_{n|n-1}\right)$$

Purpose:
This equation corrects the predicted state using the actual measurement.
Explanation:

- $\hat{\mathbf{x}}_{n|n-1}$ is the predicted state.
- $\mathbf{z}_n$ is the measurement obtained at time n.
- $\mathbf{H}\hat{\mathbf{x}}_{n|n-1}$ is the predicted measurement.
- The difference $\left(\mathbf{z}_n - \mathbf{H}\hat{\mathbf{x}}_{n|n-1}\right)$ is called the innovation or residual.
- The term $\mathbf{K}_n \left(\mathbf{z}_n - \mathbf{H}\hat{\mathbf{x}}_{n|n-1}\right)$ applies a correction based on the reliability of the measurement.
- $\hat{\mathbf{x}}_{n|n}$ is the updated and improved state estimate.

Example:
If the predicted position is 120 m and the measured position is 125 m, the innovation is 5 m. With a Kalman Gain of 0.8, the correction is $0.8 \times 5 = 4$ m, resulting in an updated position of 124 m.

Equation (13): Uncertainty Update

$$\mathbf{P}_{n|n} = (\mathbf{I} - \mathbf{K}_n\mathbf{H})\mathbf{P}_{n|n-1}(\mathbf{I} - \mathbf{K}_n\mathbf{H})^T + \mathbf{K}_n\mathbf{R}\mathbf{K}_n^T$$

Purpose:
This equation updates the uncertainty after the measurement has been incorporated.
Explanation:

- The term $(\mathbf{I} - \mathbf{K}_n\mathbf{H})\mathbf{P}_{n|n-1}(\mathbf{I} - \mathbf{K}_n\mathbf{H})^T$ reduces uncertainty due to the information gained from the measurement.
- The term $\mathbf{K}_n\mathbf{R}\mathbf{K}_n^T$ adds uncertainty caused by measurement noise.
- $\mathbf{P}_{n|n}$ is the final updated uncertainty covariance.

Why it matters:
Measurements generally reduce uncertainty, but noisy sensors introduce some uncertainty back. This equation ensures a balanced and realistic uncertainty estimate.

7.4 Matrix Dimensions for the 3D Constant Jerk Model

For the three-dimensional constant jerk motion model used in this work:

- $\mathbf{x}$: 12×1 state vector
- $\mathbf{F}$: 12×12 state transition matrix
- $\mathbf{P}$: 12×12 state covariance matrix
- $\mathbf{Q}$: 12×12 process noise covariance
- $\mathbf{H}$: 3×12 measurement matrix
- $\mathbf{R}$: 3×3 measurement noise covariance
- $\mathbf{K}$: 12×3 Kalman Gain
- $\mathbf{z}$: 3×1 measurement vector

7.5 Complete Algorithm Flow

1. Initialize the state estimate $\hat{\mathbf{x}}_{0|0}$ and covariance $\mathbf{P}_{0|0}$.
2. Predict the next state and uncertainty using Equations (9) and (10).
3. Obtain the measurement $\mathbf{z}_n$ from sensors.
4. Compute the Kalman Gain using Equation (11).
5. Update the state estimate using Equation (12).
6. Update the uncertainty using Equation (13).
7. Repeat the process for each subsequent time step.

8 Extended Kalman Filter (EKF)

8.1 When Do We Need an Extended Kalman Filter?

We need an Extended Kalman Filter (EKF) when our measurements are not linear with respect to our state.

Simple explanation: In the regular Kalman Filter, we use the equation:

$$\mathbf{z} = \mathbf{H}\mathbf{x}$$

This says "measurements = matrix × state". But what if measurements come from a more complicated function?

Real-world example: A radar doesn't give us (x, y, z) coordinates directly. It gives:

- How far away something is (range)
- What direction it's in (angles)

These measurements come from nonlinear formulas involving the position coordinates.

8.2 The Core Idea of EKF

The EKF handles nonlinear measurements using linearization:

1. Approximate the nonlinear function with a straight line (tangent line)
2. Use this straight line approximation in the Kalman filter equations
3. Recalculate the approximation at each step (since we move along the curve)

This is like using a ruler to approximate a curved road at your current location.

8.3 EKF Equations

Prediction Step (Same as Regular KF)

Equation (14): State Prediction

$$\hat{\mathbf{x}}_{n|n-1} = \mathbf{F}\hat{\mathbf{x}}_{n-1|n-1}$$

- Predicts where the object will be using our motion model
- Same as before because our motion is linear (constant jerk)

Equation (15): Uncertainty Prediction

$$\mathbf{P}_{n|n-1} = \mathbf{F}\mathbf{P}_{n-1|n-1}\mathbf{F}^T + \mathbf{Q}$$

- Predicts how uncertain we'll be about our prediction

- Adds process noise $\mathbf{Q}$ for unmodeled effects

Update Step
Equation (16): Kalman Gain

$$\mathbf{K}_n = \mathbf{P}_{n|n-1}\mathbf{H}_n^T \left(\mathbf{H}_n\mathbf{P}_{n|n-1}\mathbf{H}_n^T + \mathbf{R}\right)^{-1}$$

What's new: $\mathbf{H}_n = \left.\frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{n|n-1}}$

- Instead of constant $\mathbf{H}$, we use the Jacobian matrix
- It is the derivative of our measurement function
- It tells us how measurements change when states change
- We compute it at our current estimate $\hat{\mathbf{x}}_{n|n-1}$

Equation (17): State Update

$$\hat{\mathbf{x}}_{n|n} = \hat{\mathbf{x}}_{n|n-1} + \mathbf{K}_n \left[\mathbf{z}_n - \mathbf{h}(\hat{\mathbf{x}}_{n|n-1})\right]$$

What's new: $\mathbf{h}(\hat{\mathbf{x}}_{n|n-1})$ instead of $\mathbf{H}\hat{\mathbf{x}}_{n|n-1}$

- We use the actual nonlinear function $\mathbf{h}(\cdot)$ to predict what we should measure
- $\mathbf{z}_n - \mathbf{h}(\hat{\mathbf{x}}_{n|n-1})$ is the difference between actual and predicted measurements

Equation (18): Uncertainty Update

$$\mathbf{P}_{n|n} = (\mathbf{I} - \mathbf{K}_n\mathbf{H}_n)\mathbf{P}_{n|n-1}(\mathbf{I} - \mathbf{K}_n\mathbf{H}_n)^T + \mathbf{K}_n\mathbf{R}\mathbf{K}_n^T$$

- Updates our uncertainty after incorporating the measurement
- Uses the Jacobian $\mathbf{H}_n$ instead of constant $\mathbf{H}$

8.4 When to Use EKF vs Regular KF

Use Regular Kalman Filter when:

- Measurements are linear functions of states (like direct x, y, z coordinates)
- Equation is: $\mathbf{z} = \mathbf{H}\mathbf{x}$

Use Extended Kalman Filter when:

- Measurements are nonlinear functions (like range, angles, pixel coordinates)
- Equation is: $\mathbf{z} = \mathbf{h}(\mathbf{x})$ where $\mathbf{h}$ is nonlinear

For our project:

- If sensors give (x, y, z) directly $\rightarrow$ we will use Regular KF
- If sensors give (range, azimuth, elevation) $\rightarrow$ we will use EKF

8.5 Summary

The Extended Kalman Filter extends the regular Kalman Filter to handle nonlinear measurements by:

1. Linearizing the measurement function using its derivative (Jacobian)
2. Using this linear approximation in the Kalman equations
3. Recalculating the linearization at each time step

This allows us to use sensors that don't give simple linear measurements while still benefiting from the Kalman filter's optimal estimation properties (at least approximately).

The key change from LKF to EKF is replacing the constant $\mathbf{H}$ matrix with the Jacobian $\mathbf{H}_n$, recomputed at each step based on our current estimate.

9 Nonlinear Measurement Model

9.1 What Are Spherical Coordinates?

When a radar or similar sensor looks at an object in 3D space, it doesn't tell us the object's (x, y, z) coordinates directly. Instead, it gives us three measurements in spherical coordinates:

$$\mathbf{z} = \begin{bmatrix} r \\ \theta \\ \phi \end{bmatrix}$$

These three numbers describe where the object is in a different way than (x, y, z).

9.2 Range (r) – "How Far Away"

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2}$$

What it means: This is simply how far the object is from the sensor.

Think of it like: If I am pointing at something and someone asks "How far is it?", I will give a distance like "100 meters". That's the range.

In math terms: It's the straight-line distance from the sensor (usually at origin) to the object. We calculate it using the Pythagorean theorem in 3D.

9.3 Azimuth (θ) – "Which Horizontal Direction"

$$\theta = \text{arctan2}(p_y, p_x)$$

What it means: This tells us which way to look horizontally to see the object.

Think of it like: If you're facing north, and the object is to your right, that's an azimuth of 90° (east). If it's behind you, that's 180° (south), etc.

Key points:

- Usually measured in degrees or radians
- Goes from 0° to 360° (full circle)
- 0° is typically north or the sensor's forward direction
- Uses arctan2 instead of regular arctan to handle all four quadrants correctly

9.4 Elevation (ϕ) – "How High Up or Down"

$$\phi = \text{arctan2}\left(p_z, \sqrt{p_x^2 + p_y^2}\right)$$

What it means: This tells us how high above or below the horizontal plane the object is.

Think of it like:

- Looking straight ahead (horizontal) = 0° elevation
- Looking up at the sky = positive elevation (like +30°)
- Looking down at the ground = negative elevation (like −10°)

Important: The formula uses the horizontal distance $\sqrt{p_x^2 + p_y^2}$ in the denominator, not the full 3D distance. This gives us the correct angle above/below the horizon.

9.5 Putting It All Together

Real-world example: Air traffic control radar:

- Range: "The plane is 50 km away"
- Azimuth: "It's at bearing 120° (southeast)"
- Elevation: "It's at 5° above the horizon"

Why not just use (x, y, z)?

1. Radars naturally measure this way: They send out pulses and measure how long they take to return (range) and what direction they came from (angles)
2. Direct measurements: These are what the sensor actually reads
3. Noisy in different ways: Range noise is different from angle noise

9.6 The Problem for Our Kalman Filter

Our Kalman filter works with (x, y, z) coordinates in its state, but our sensor gives us (r, θ, ϕ).

So we need to convert between these two coordinate systems. The measurement function $\mathbf{h}(\mathbf{x})$ does exactly this conversion.

The conversion formulas are nonlinear. That's why we need the Extended Kalman Filter. We can't write these measurements as a simple matrix multiplication $\mathbf{Hx}$.

9.7 Visual Example

Imagine a drone flying:

- Position in (x, y, z): $(100\,\text{m}, 50\,\text{m}, 30\,\text{m})$

In spherical coordinates from a ground radar:

- Range: $r = \sqrt{100^2 + 50^2 + 30^2} = 116.6\,\text{m}$
- Azimuth: $\theta = \text{arctan2}(50, 100) = 26.6°$
- Elevation: $\phi = \text{arctan2}(30, \sqrt{100^2 + 50^2}) = 15.5°$

These three numbers $(116.6, 26.6°, 15.5°)$ are what the radar actually gives us, and we need to work with them in our filter.

10 Jacobian Matrix

10.1 What is the Jacobian and Why Do We Need It?

10.1.1 The Problem We're Solving

Our measurement function $\mathbf{h}(\mathbf{x})$ converts from (x, y, z) coordinates to (r, θ, ϕ):

$$\mathbf{h}(\mathbf{x}) = \begin{bmatrix} \sqrt{p_x^2 + p_y^2 + p_z^2} \\ \text{arctan2}(p_y, p_x) \\ \text{arctan2}\left(p_z, \sqrt{p_x^2 + p_y^2}\right) \end{bmatrix}$$

This function is nonlinear. We can't write it as $\mathbf{z} = \mathbf{Hx}$.

10.1.2 The Jacobian Solution

The Jacobian is a matrix of partial derivatives that gives us the best linear approximation of $\mathbf{h}(\mathbf{x})$ near our current estimate.

Simple analogy: If you're on a curved road, the Jacobian is like telling you "right now, for every meter you go east, the road curves 0.1 meters north." It's the local slope of the function.

10.2 Deriving the Jacobian Step-by-Step

10.2.1 Step 1: Understanding the Jacobian Structure

Our state has 12 variables:

$$\mathbf{x} = [p_x, v_x, a_x, j_x, p_y, v_y, a_y, j_y, p_z, v_z, a_z, j_z]^T$$

Our measurements are 3 variables: r, θ, ϕ.
So the Jacobian $\mathbf{H}$ will be a 3×12 matrix.
Important fact: Measurements only depend on position, not velocity, acceleration, or jerk.
So:

- All derivatives with respect to $v_x, a_x, j_x, v_y, a_y, j_y, v_z, a_z, j_z$ are zero
- Only derivatives with respect to p_x, p_y, p_z are nonzero

10.2.2 Step 2: Range (r) Derivatives

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2}$$

Partial derivative with respect to p_x:

$$\frac{\partial r}{\partial p_x} = \frac{p_x}{\sqrt{p_x^2 + p_y^2 + p_z^2}} = \frac{p_x}{r}$$

(This comes from: derivative of $\sqrt{u}$ is $\frac{1}{2\sqrt{u}} \times 2p_x = \frac{p_x}{\sqrt{u}}$)
Similarly:

$$\frac{\partial r}{\partial p_y} = \frac{p_y}{r}, \quad \frac{\partial r}{\partial p_z} = \frac{p_z}{r}$$

All other derivatives of r are zero (doesn't depend on velocity, etc.)
So **Row 1 of Jacobian** (for range):

$$\left[\frac{p_x}{r}, 0, 0, 0, \frac{p_y}{r}, 0, 0, 0, \frac{p_z}{r}, 0, 0, 0\right]$$

10.2.3 Step 3: Azimuth (θ) Derivatives

Recall: $\arctan\left(\frac{y}{x}\right) = \frac{\partial}{\partial x}\left[-\frac{y}{x^2+y^2}\right]$
So:

$$\theta = \text{arctan2}\left(\frac{p_y}{p_x}\right)$$

$$\frac{\partial \theta}{\partial p_x} = -\frac{p_y}{p_x^2 + p_y^2}$$

$$\frac{\partial \theta}{\partial p_y} = \frac{p_x}{p_x^2 + p_y^2}$$

$$\frac{\partial \theta}{\partial p_z} = 0 \quad \text{(azimuth doesn't depend on height!)}$$

All other derivatives are zero.
So **Row 2 of Jacobian** (for azimuth):

$$\left[-\frac{p_y}{p_x^2 + p_y^2}, 0, 0, 0, \frac{p_x}{p_x^2 + p_y^2}, 0, 0, 0, 0, 0, 0, 0\right]$$

10.2.4 Step 4: Elevation (ϕ) Derivatives

$$\phi = \arctan2\left(\frac{p_z}{\sqrt{p_x^2 + p_y^2}}\right)$$

Let $\rho = \sqrt{p_x^2 + p_y^2}$ = horizontal distance
Derivative with respect to p_x:

$$\frac{\partial \phi}{\partial p_x} = \frac{\partial}{\partial p_x} \arctan\left(\frac{p_z}{\rho}\right)$$

Using chain rule: $\arctan(u)' = \frac{1}{1+u^2}$
First, $u = \frac{p_z}{\rho}$, so $\frac{\partial u}{\partial p_x} = p_z \cdot \frac{\partial}{\partial p_x}\left(\frac{1}{\rho}\right) = -\frac{p_z p_x}{\rho^3}$
(since $\frac{\partial \rho}{\partial p_x} = \frac{p_x}{\rho}$)
Then $\frac{\partial \phi}{\partial p_x} = \frac{1}{1+\left(\frac{p_z}{\rho}\right)^2} \cdot \left(-\frac{p_z p_x}{\rho^3}\right) = \frac{1}{\frac{\rho^2+p_z^2}{\rho^2}} \cdot \left(-\frac{p_z p_x}{\rho^3}\right)$
But $\rho^2 + p_z^2 = r^2$, and $\rho^2 r^2 = \rho^2(p_x^2 + p_y^2 + p_z^2)$
So: $\frac{\partial \phi}{\partial p_x} = -\frac{p_x p_z}{r^2 \rho}$
Similarly:

$$\frac{\partial \phi}{\partial p_y} = -\frac{p_y p_z}{r^2 \sqrt{p_x^2 + p_y^2}}$$

$$\frac{\partial \phi}{\partial p_z} = \frac{\sqrt{p_x^2 + p_y^2}}{r^2} = \frac{\rho}{r^2}$$

All other derivatives are zero.
So **Row 3 of Jacobian** (for elevation):

$$\left[-\frac{p_x p_z}{r^2 \rho}, 0, 0, 0, -\frac{p_y p_z}{r^2 \rho}, 0, 0, 0, \frac{\rho}{r^2}, 0, 0, 0\right]$$

where $\rho = \sqrt{p_x^2 + p_y^2}$

10.3 Complete Jacobian Matrix

Putting it all together:

$$\mathbf{H} = \frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{p_x}{r} & 0 & 0 & 0 & \frac{p_y}{r} & 0 & 0 & 0 & \frac{p_z}{r} & 0 & 0 & 0 \\ -\frac{p_y}{\rho^2} & 0 & 0 & 0 & \frac{p_x}{\rho^2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -\frac{p_x p_z}{r^2 \rho} & 0 & 0 & 0 & -\frac{p_y p_z}{r^2 \rho} & 0 & 0 & 0 & \frac{\rho}{r^2} & 0 & 0 & 0 \end{bmatrix}$$

Where:

- $r = \sqrt{p_x^2 + p_y^2 + p_z^2}$ (3D distance)
- $\rho = \sqrt{p_x^2 + p_y^2}$ (horizontal distance)

Important: We evaluate this at $\mathbf{x} = \hat{\mathbf{x}}_{n|n-1}$, our current predicted state.

10.4 Why the Jacobian is Required

1. **Linearization for Kalman Equations:**

$$\mathbf{h}(\mathbf{x}) \approx \mathbf{h}(\hat{\mathbf{x}}) + \mathbf{H}(\mathbf{x} - \hat{\mathbf{x}})$$

2. **Tells How Measurements Change:**
   Each entry of **H** indicates how a small change in state affects measurement.
   Example:

$$\frac{\partial r}{\partial p_x} = \frac{p_x}{r}$$

   - If the object moves straight ahead, range changes proportionally.
   - If it moves sideways, range does not change.

3. **Kalman Gain Calculation:**

$$\mathbf{K} = \mathbf{P}_{n|n-1}\mathbf{H}^T \left(\mathbf{H}\mathbf{P}_{n|n-1}\mathbf{H}^T + \mathbf{R}\right)^{-1}$$

4. **Local Linear Approximation:**
   H is local, recomputed at each time step, so the EKF can handle nonlinear measurements dynamically.

article xcolor

Conclusion

In this milestone, we built the mathematical foundation for applying Kalman filtering to three-dimensional maneuvering targets. We derived the discrete-time constant jerk model and obtained the 12×12 state transition matrix (F) and process noise covariance matrix (Q) by integrating the continuous-time motion equations.

We explained how the Linear Kalman Filter works when measurements are given in Cartesian coordinates. For nonlinear spherical measurements (such as those from radar or lidar), we used the Extended Kalman Filter (EKF). The Jacobian matrix was derived to linearize the measurement function, allowing accurate estimation despite the nonlinear relationship between the state and measurements.

All results were developed from first principles with clear physical meaning, making the framework both correct in theory and ready for practical implementation in the next stage.

References

[1] Alex Becker, Kalman Filter from the Ground Up, 2023.

[2] Li and Jilkov, A Jerk Model for Tracking Highly Maneuvering Targets.

[3] NASA Technical Report, "A Discrete-Time Constant Jerk Tracking Model for Maneuvering Targets", 2018.

[4] MathWorks, Introduction to Kalman Filters for Object Tracking.